

Same
Data
Bigger
Possibilities

{JSON} for DevOps BEGINNERS

Data
Drives
DevOps

From JSON Fundamentals to
Real DevOps Automation and Configuration



APIs



Cloud



Kubernetes



Automation



CI/CD



Monitoring

```
1 {  
2   "name": "devops",  
3   "learn": true,  
4   "build": "automation",  
5   "succeed": "together"  
6 }
```

- ✓ Practical Examples
- ✓ Real DevOps Use Cases
- ✓ Step-by-Step Learning
- ✓ Tools and Commands
- ✓ Production Best Practices

Structured Data
Stronger DevOps

Small Syntax
Big Impact



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Tools
People
Process
Progress

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Systems

Same
Data
Bigger
Possibilities

JSON EXPLAINED, WHY DEVOPS ENGINEERS USE IT

JSON
Drives
DevOps

1 WHAT JSON IS

- JSON (JavaScript Object Notation) is a lightweight, text-based format for storing and exchanging data.
- It uses a simple key-value structure that is easy to read and write.
- JSON is language-independent. It can be used by any tool, platform or programming language.

2 WHY IT EXISTS

- Computers and applications need a common way to share structured data.
- JSON was created to make data exchange simple, consistent and human-readable.
- It replaced older, more complex formats like XML in many modern systems.

3 HUMAN-READABLE STRUCTURED DATA

- JSON organizes data in a clear structure using keys and values.
- It is easy for humans to read and easy for machines to process.
- This makes it ideal for configuration, APIs, and automation.

```
1 {
2   "name": "veriqta",
3   "type": "devops",
4   "active": true,
5   "version": "1.0"
}
```

4 JSON IN DEVOPS

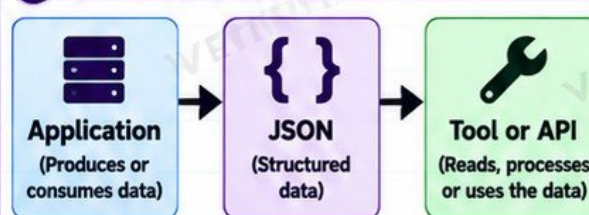
- APIs return data in JSON (AWS, Azure, GCP, GitHub, etc.).
- Cloud tools use JSON for input and output.
- Kubernetes and container tools use JSON for configuration and status.
- CI/CD systems use JSON for pipeline data and integration.
- Automation scripts use JSON to read, process, and exchange data.



5 JSON IS DATA, NOT A PROGRAMMING LANGUAGE

- JSON is a data format, not a programming language.
- You do not run JSON. You read, write, parse, or send it.
- Many programming languages and tools work with JSON.
- It is simply a way to structure information.

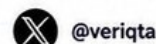
6 MENTAL MODEL



7 JUNIOR ENGINEER TIP



Learn to read structure before memorizing syntax.
Understanding how JSON is organized will help you work with it confidently in real DevOps scenarios.



Better
Engineers
Brighter
Systems

VERIQTA

LEARN | PRACTICE | BUILD | GROW

DevOps
Today
A Stronger
Tomorrow

Same
Data
Bigger
Possibilities

{ JSON STRUCTURE }

OBJECTS, KEYS, AND VALUES

Practice
Today
Stronger
Tomorrow

JSON uses a simple and consistent structure made up of objects, keys, and values. Understanding this structure is the foundation for working with JSON in real DevOps scenarios.

1 OBJECTS

- An object is a collection of key-value pairs.
- It is enclosed in curly braces {}.
- Objects can contain strings, numbers, booleans, arrays or other objects.

```
{
  "name": "veriqta",
  "type": "devops"
}
```

2 KEY-VALUE PAIRS

- Data in JSON is stored as key-value pairs.
- The key is a name (always a string).
- The value can be a string, number, boolean, array, object, or null.

```
{
  "key": "value",
  "enabled": true,
  "port": 8080
}
```

3 CURLY BRACES

- Curly braces {} define an object.
- Everything inside the braces is part of the same object.
- Objects can be nested inside other objects.

```
{
  "app": {
    "name": "veriqta",
    "version": "1.0"
  }
}
```

4 COLONS AND COMMAS

- A colon : separates a key from its value.
- A comma , separates multiple key-value pairs.
- The last key-value pair does not have a comma.

```
{
  "name": "veriqta",
  "type": "devops",
  "active": true
}
```

5 QUOTED KEYS

- Keys must always be in double quotes " ".
- Values that are strings must also be in double quotes.
- This is required in JSON (unlike some other formats like YAML).

```
{
  "name": "veriqta",
  "environment": "production"
}
```

6 SIMPLE EXAMPLE

- A basic JSON object with different value types.

```
{
  "name": "veriqta",
  "version": "1.0",
  "active": true,
  "port": 8080,
  "tags": ["devops", "cloud"],
  "metadata": {
    "team": "platform"
  }
}
```

7 HOW ENGINEERS MENTALLY SCAN JSON

- Look at the keys first to understand the structure.
- Identify the main sections (e.g., metadata, spec, status).
- Expand nested objects only when needed.
- Focus on the specific field you need.
- Ignore the rest to avoid information overload.



Engineers learn to quickly scan keys, look for familiar section names, and find the exact value they need without reading everything line by line.

8 COMMON MISTAKE: MISSING COMMAS

- Forgetting a comma between key-value pairs will make the JSON invalid.
- This is one of the most common errors for beginners.

```
{
  "name": "veriqta"
  "type": "devops"
}
```

Invalid JSON

Missing comma after the first key-value pair.

```
{
  "name": "veriqta",
  "type": "devops"
}
```

Valid JSON

Comma added. Now the JSON is correct.

{JSON} DATA TYPES

Understand the building blocks of JSON data

1 STRING

- A sequence of characters (text).
- Always written in double quotes.

Example:

```
"name": "veriqta"
```

2 NUMBER

- A numeric value. Can be an integer or a decimal.
- No quotes.

Example:

```
"port": 8080
"version": 1.5
```

3 BOOLEAN

- A true or false value.
- Used for conditions, feature flags, etc.
- No quotes.

Example:

```
"enabled": true
"debug": false
```

4 NULL

- Represents no value or an empty value.
- Used when a value is unknown or not set.
- No quotes.

Example:

```
"description": null
```

5 OBJECT

- A collection of key-value pairs.
- Enclosed in curly braces { }.
- Used to represent structured data.

```
"user": {
  "name": "veriqta",
  "role": "devops",
  "active": true
}
```

6 ARRAY

- An ordered list of values.
- Enclosed in square brackets [].
- Can contain strings, numbers, objects, or other arrays.

```
"servers": [
  "web-1",
  "web-2",
  "web-3",
]
```

7 VALID vs INVALID VALUES



VALID JSON

```
"name": "app"
"port": 8080
"active": true
"count": 0
"data": null
"items": ["a", "b"]
"config": {}
```



INVALID JSON

```
"name": 'app'
(single quotes)

"port": "8080"
(number in quotes)

"active": True
(capital T)

"items": [1,2,]
(trailing comma)

{name: "app"}
(unquoted key)
```

8 WHY DATA TYPE MATTERS

- Tools and APIs expect the correct data type.
- Wrong data types can cause errors in automation scripts, CI/CD pipelines, and cloud configurations.
- For example, a port must be a number, not a string.
- Booleans control behavior in infrastructure tools.
- Consistent data types make your configuration reliable and predictable.

9 REAL DEVOPS EXAMPLES

- AWS CLI returns JSON with strings, numbers, and arrays.
- Kubernetes manifests use JSON objects and arrays.
- CI/CD tools use JSON for pipeline configuration.
- Monitoring tools (e.g. Prometheus, Grafana) return JSON data.
- Infrastructure as Code (Terraform) outputs data in JSON.
- Automation scripts (Python, Bash, Node.js) parse JSON to make decisions.

10 JUNIOR ENGINEER TIP



Learn to read the structure and understand the data types before memorizing syntax. Once you understand the types, JSON becomes easy to read, use, and troubleshoot in real DevOps work.

ARRAYS AND LISTS IN JSON

A collection of values in a specific order

1 SQUARE BRACKETS

- Arrays are written using square brackets `[ ]`.
- They can hold multiple values.
- Values are separated by commas.

```
1 [
2   "web-1", "web-2", "web-3"
3 ]
```

2 MULTIPLE VALUES

- An array can contain many values of the same or different types.
- Values are ordered (position matters).

```
1 [
2   "app", 8080, true, null
3 ]
```

3 ARRAYS OF STRINGS

- A list of text values.
- Each value is in double quotes.

```
1 [
2   "nginx",
3   "apache",
4   "tomcat"
5 ]
```

4 ARRAYS OF OBJECTS

- Arrays can contain objects (key-value pairs).
- Useful for configurations like servers, users, or services.

```
1 [
2   { "name": "web-1", "port": 80 },
3   { "name": "web-2", "port": 80 }
4 ]
```

5 NESTED ARRAYS

- An array can contain another array.
- Useful for grouped or multi-dimensional data.

```
1 [
2   ["us-east-1", "us-west-2"],
3   ["web", "app", "db"]
4 ]
```

6 REAL DEVOPS EXAMPLE

Example: servers, regions and security groups.

```
1 {
2   "servers": ["web-1", "web-2"],
3   "regions": ["us-east-1", "us-west-2"],
4   "securityGroups": [
5     "sg-12345",
6     "sg-67890",
7     "sg-abcde"
8   ]
9 }
```

7 BEGINNER MISTAKE



Confusing arrays with objects.

- Objects use curly braces `{ }` and have key-value pairs.
- Arrays use square brackets `[ ]` and contain a list of values.
- Do not use key names inside an array.

INVALID (Array with keys)

```
1 [
2   "name": "web-1",
3   "name": "web-2"
4 ]
```

Arrays cannot have keys like this.

VALID (Array of objects)

```
1 [
2   { "name": "web-1" },
3   { "name": "web-2" }
4 ]
```

Each item is an object inside the array.

8 KEY TAKEAWAYS

- ✓ Arrays use square brackets `[ ]`.
- ✓ They store multiple values.
- ✓ Arrays can contain strings, numbers, booleans, null, objects, or other arrays.
- ✓ Widely used in DevOps for configurations, APIs, cloud resources, and automation.
- ✓ Understand the difference between arrays and objects.
- ✓ Always check the structure before using JSON in scripts or tools.

{ NESTED JSON }

HOW REAL CONFIGURATIONS ARE STRUCTURED

Real-world data is nested. Understand it, and you can work with any DevOps tool.

Same
Data
Bigger
Possibilities

1 OBJECTS INSIDE OBJECTS

- A JSON object can contain another object.
- Used to group related data.
- Common in APIs, cloud configs, and Kubernetes.

```
{
  "name": "web-server",
  "metadata": {
    "env": "prod",
    "team": "devops"
  }
}
```

The `metadata` key contains another object.

2 ARRAYS INSIDE OBJECTS

- An object can contain an array.
- Used for lists like security groups, tags, or ports.
- Very common in DevOps tools.

```
{
  "name": "web-server",
  "ports": [
    80,
    443,
    8080
  ]
}
```

The `ports` key contains an array of values.

3 OBJECTS INSIDE ARRAYS

- An array can contain multiple objects.
- Used for lists of servers, users, or resources.
- Common in cloud and Kubernetes.

```
{
  "servers": [
    { "name": "web-1", "az": "us-east-1" },
    { "name": "web-2", "az": "us-west-2" }
  ]
}
```

The `servers` key contains an array of objects.

4 READING FROM OUTSIDE INWARD

- Start from the outermost level.
- Move inward step by step.
- Each level narrows down the data.
- Helps you find the exact value you need.

Path: `servers[0].name`



5 DOT-STYLE MENTAL MODEL

- Use dot notation to think about the structure.
- Each dot moves one level deeper.
- Array items use an index like `[0]`.
- This is the same style used in tools like `jq`, `JSONPath`, and many CLIs.

Example paths:

```

metadata.env      → Access the env value
servers[0].name   → First server name
servers[1].az     → Second server AZ
network.securityGroups[0] → First security group
  
```

6 EXAMPLE: CLOUD INSTANCE CONFIGURATION

A real-world style example of a cloud instance configuration.

```
{
  "instance": {
    "name": "web-server",
    "type": "t3.medium",
    "region": "us-east-1",
    "tags": [
      { "key": "Environment", "value": "Production" },
      { "key": "Team", "value": "DevOps" }
    ],
    "network": {
      "securityGroups": [
        "sg-12345", "sg-67890"
      ]
    }
  }
}
```

7 TROUBLESHOOTING DEEPLY NESTED DATA

- Check the structure carefully (braces, brackets, commas).
- Use a JSON formatter (`jq`, online tools, or your editor).
- Verify the correct path (especially array indexes).
- Look for missing keys or null values.
- Use tools like `jq`, `kubectl`, or `AWS CLI` to explore the data.
- Print sections step by step to find the issue.

COMMON ISSUES

- ✗ Wrong path (e.g. `servers.name`)
- ✗ Array index out of range
- ✗ Missing key
- ✗ Null value
- ✗ Expecting an object but got an array
- ✗ Syntax errors (missing comma or brace)



JUNIOR ENGINEER TIP

Nested JSON may look complex at first, but it's just objects and arrays. Take it one level at a time. Use a JSON formatter, explore the structure, and practice reading real outputs from cloud and DevOps tools.

{ VALID JSON RULES }

EVERY DEVOPS ENGINEER MUST KNOW

Small syntax. Big impact. Get it right.

1 DOUBLE QUOTES

- Keys and string values must use double quotes (").
- Single quotes (') are not valid in standard JSON.

VALID ✓

```
{
  "name": "veriqta"
}
```

INVALID ✗

```
{
  'name': 'veriqta'
}
```

Use double quotes for all keys and string values.

2 NO TRAILING COMMAS

- Do not put a comma after the last item in an object or array.

VALID ✓

```
{
  "name": "app"
}
```

INVALID ✗

```
{
  "name": "app",
}
```

No comma after the last item.

3 NO COMMENTS

- Standard JSON does not support comments.
- Comments like // or /* */ will cause a parsing error.

VALID ✓

```
{
  "name": "app"
}
```

INVALID ✗

```
{
  // this is a comment
  "name": "app"
}
```

Do not add comments in JSON files.

4 PROPER BRACES AND BRACKETS

- Use {} for objects.
- Use [] for arrays.
- Make sure they are correctly opened and closed.

VALID ✓

```
{
  "items": [1, 2, 3]
}
```

INVALID ✗

```
{
  "items": [1, 2, 3]
}
```

Always close your braces and brackets.

5 CORRECT COMMAS

- Use commas between items in objects and arrays.
- Do not miss commas.

VALID ✓

```
{
  "a": 1,
  "b": 2
}
```

INVALID ✗

```
{
  "a": 1
  "b": 2
}
```

Separate each item with a comma.

6 VALID BOOLEANS

- Booleans must be lowercase.
- Use true or false only.

VALID ✓

```
{
  "enabled": true,
  "debug": false
}
```

INVALID ✗

```
{
  "enabled": True,
  "debug": FALSE
}
```

Use lowercase true and false.

7 null

- Use null to represent no value.
- Must be lowercase.

VALID ✓

```
{
  "value": null
}
```

INVALID ✗

```
{
  "value": NULL
}
```

Use lowercase null.

8 WHY SMALL SYNTAX ERRORS BREAK PIPELINES

- Tools like jq, Kubernetes, Terraform, AWS CLI, and CI/CD systems expect valid JSON.
- A small syntax error can cause parsing failures.
- Invalid JSON can stop deployments, break automation, or return incomplete data.



VALID JSON KEEPS
YOUR PIPELINES RUNNING.

9 VALID vs INVALID EXAMPLES

VALID JSON ✓

```
{
  "name": "web-app",
  "replicas": 3,
  "enabled": true,
  "tags": ["dev", "prod"],
  "config": { "env": null }
}
```

INVALID JSON ✗

```
{
  "name": 'web-app', // single quotes
  "replicas": 3,    // missing comma
  "enabled": true,   // capital True
  "tags": ["dev", "prod", ], // trailing comma
  "config": { "env": NULL } // capital NULL
}
```



JUNIOR ENGINEER TIP

Always validate your JSON before using it in scripts, APIs, or pipelines.
A correct structure today saves you hours of debugging tomorrow.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

VERI Q T A

LEARN | PRACTICE | BUILD | GROW

JSON vs YAML

Same data, different syntax

Two formats. One goal. Structured data.

Same
Data
Bigger
Possibilities

1 SAME DATA, DIFFERENT SYNTAX

- JSON and YAML can represent the same data.
- They store the same information, but use different syntax and rules.
- You can convert between them.

JSON

```
{
  "name": "web-app",
  "replicas": 3,
  "enabled": true
}
```

YAML

```
name: web-app
replicas: 3
enabled: true
```

Same data. Different syntax. Same meaning.

2 JSON IS EXPLICIT

- Uses curly braces `{ }` and quotes `" "`.
- Keys and values must be in double quotes.
- Stricter format.
- Less flexible, but very consistent.
- Commonly used in APIs and automation tools.



JSON

- ✓ Explicit
- ✓ Strict syntax
- ✓ Great for APIs
- ✓ Machine-friendly

3 YAML IS MORE HUMAN-FRIENDLY

- Uses indentation instead of braces.
- No quotes required for simple values.
- Easier to read and write.
- More flexible.
- Widely used for configuration files, especially in Kubernetes and DevOps.



YAML

- ✓ Easy to read
- ✓ Clean syntax
- ✓ Great for configs
- ✓ Human-friendly

4 KUBERNETES CONNECTION

- Kubernetes accepts both JSON and YAML.
- Most Kubernetes manifests are written in YAML.
- You can also use JSON if needed (e.g. with automation or APIs).



kubernetes

YAML is more common for Kubernetes because it is easier to read and manage.

5 APIs OFTEN RETURN JSON

- Most APIs (AWS, Azure, GCP, GitHub, etc.) return data in JSON format.
- JSON is easy for machines to parse.
- You will frequently work with JSON when using:
 - REST APIs
 - Cloud CLI tools
 - Monitoring tools
 - CI/CD systems



APIs speak JSON because it is structured, consistent, and machine-friendly.

6 CONFIGURATION OFTEN USES YAML

- Configuration files are commonly written in YAML.
- Examples:
 - Kubernetes manifests
 - Docker Compose files
 - CI/CD pipeline files (GitHub Actions, GitLab CI)
 - Infrastructure as Code (e.g. Ansible, Helm, Terraform)
 - Application configuration files



YAML is popular for configuration because it is simple, clean, and easy to maintain.

7 WHEN ENGINEERS USE EACH

Use JSON when:

- Working with APIs
- Using cloud CLI tools
- Automation and scripting
- Machine-to-machine communication

Use YAML when:

- Writing Kubernetes manifests
- Configuration files
- Infrastructure as Code
- CI/CD pipeline files
- Human-readable configs

8 WHY KNOWING BOTH MATTERS

- DevOps engineers work with both JSON and YAML daily.
- They are used in different tools and scenarios.
- Understanding both makes you more flexible and effective.
- You can debug issues faster.
- You can move between tools, platforms, and environments easily.
- It is an essential skill for real-world DevOps work.



KNOWING BOTH FORMATS GIVES YOU A BIG ADVANTAGE IN REAL DEVOPS ENVIRONMENTS.



JUNIOR ENGINEER TIP

Learn both JSON and YAML. Use the right tool for the job. The more you work with them, the more natural they become.

JSON vs XML

AND OTHER STRUCTURED DATA FORMATS

Understand the differences and when to use each

1 WHAT IS STRUCTURED DATA?

- Structured data is information organized in a defined format with clear structure.
- It uses rules, keys, tags, or separators so machines can easily read, process, and exchange data.
- Examples: JSON, XML, YAML, CSV.
- Structured data makes integration, automation, and data sharing possible in DevOps and modern applications.

2 WHY APIS COMMONLY PREFER JSON

- JSON is lightweight and easy to read.
- It has a simple, clean syntax.
- It is smaller in size compared to XML.
- It maps naturally to modern programming languages.
- It is widely supported in web, cloud, and mobile applications.

3 WHY OLDER ENTERPRISE SYSTEMS STILL USE XML

- XML was designed for large enterprise systems.
- It supports complex document structures.
- It includes strict schemas (XSD) for validation.
- It has been used for many years in industries such as finance, government, and healthcare.
- Many legacy systems still rely on XML and are not easy to replace.

JSON

Lightweight, easy to read, widely used in APIs.

```
{
  "name": "veriqta",
  "type": "devops",
  "active": true
}
```

XML

More verbose, used in enterprise systems.

```
<user>
  <name>veriqta</name>
  <type>devops</type>
  <active>true</active>
</user>
```

YAML

Human-friendly, commonly used in DevOps tools.

```
name: veriqta
type: devops
active: true
```

CSV

Simple tabular format, good for basic data.

```
name,type,active
veriqta,devops,true
```

4 PRACTICAL COMPARISON

FEATURE	JSON	XML	YAML	CSV
Syntax style	Simple	Verbose	Human-friendly	Very simple
Readability	High	Lower	Very high	High (simple data)
File size	Small	Larger	Small	Small
Supports complex data	Yes	Yes (very strong)	Yes	No (flat data only)
Schema/validation	Limited	Strong (XSD)	Limited	None
Common use cases	APIs, cloud, DevOps	Enterprise systems, legacy integrations	Kubernetes, configuration files	Reports, simple datasets
Human editable	Yes	Harder	Yes	Yes
Widely used in DevOps	Yes	Sometimes	Yes	Sometimes



JUNIOR ENGINEER TIP

Choose the right format for the job. In modern DevOps, you will use JSON and YAML most often, but it is still important to understand XML and CSV because you will encounter them in real systems.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Better
Engineers
Brighter
Systems

VERIQTA

LEARN | PRACTICE | BUILD | GROW

DevOps
Today
A Stronger
Tomorrow

READING JSON FROM APIs

Understand API responses and how to work with JSON data

1 WHAT AN API RESPONSE LOOKS LIKE

- An API (Application Programming Interface) lets you request data from a server.
- The server processes your request and returns a response, usually in JSON format.
- The response includes a status code, headers, and a body (JSON data).
- APIs are used everywhere in DevOps (cloud providers, monitoring tools, CI/CD, etc.).

2 HTTP REQUEST AND RESPONSE



3 STATUS CODE

- Status codes tell you if the request was successful.
- **200 OK** – Request successful
- **201 Created** – Resource created
- **400 Bad Request** – Invalid request
- **401 Unauthorized** – Authentication required
- **403 Forbidden** – You don't have permission
- **404 Not Found** – Resource not found
- **500 Internal Server Error** – Server issue

4 HEADERS

- Headers contain metadata about the request and response.
- Common response headers:
 - **Content-Type:** application/json
 - **Date:** Tue, 26 Aug 2025 10:00:00 GMT
 - **Server:** nginx
 - **Content-Length:** 512
 - **X-RateLimit-Remaining:** 4999
 - **X-Request-Id:** abc123xyz

5 JSON BODY

- The body contains the actual data, usually in JSON format.
- JSON is structured, making it easy to read and process.
- The exact structure depends on the API.
- You need to find the specific field you're interested in.

6 EXAMPLE API RESPONSE

```

HTTP/1.1 200 OK
Content-Type: application/json
Date: Tue, 26 Aug 2025 10:00:00 GMT
Server: nginx
Content-Length: 512
X-RateLimit-Remaining: 4999
X-Request-Id: abc123xyz
{
  "id": 12345,
  "name": "veriqta",
  "type": "organization",
  "public": true,
  "followers": 15000,
  "created_at": "2022-01-15T10:00:00Z",
  "url": "https://api.github.com/orgs/veriqta"
}
  
```

Labels in the image: Status Line (points to HTTP/1.1 200 OK), Headers (points to Content-Type, Date, Server, Content-Length, X-RateLimit-Remaining, X-Request-Id), JSON Body (points to the JSON object).

7 FINDING THE FIELD YOU ACTUALLY NEED

- API responses can be large. You usually only need a specific field.
- Tools like **jq** (Linux/Mac) help you extract fields easily.

```

# Get the organization name
curl -s https://api.github.com/orgs/veriqta | jq .name

# Get the number of followers
curl -s https://api.github.com/orgs/veriqta | jq .followers

# Get multiple fields
curl -s https://api.github.com/orgs/veriqta | \
jq '{name: .name, followers: .followers, created: .created_at}'
  
```

8 DEVOPS CONNECTION: MONITORING AND CLOUD APIs



- **Cloud Providers:** AWS, Azure, GCP all use JSON APIs.
- **Monitoring Tools:** Datadog, Prometheus, Grafana, New Relic use JSON for metrics and alerts.
- **CI/CD:** GitHub, GitLab, Jenkins expose JSON APIs for pipelines, builds, and deployments.
- **Automation:** Scripts and tools use JSON to read, process, and exchange data.
- Understanding API JSON responses helps you troubleshoot, automate, and integrate systems.



APIs + JSON
=
Automation
+
Better Operations



JUNIOR ENGINEER TIP

Start by reading and understanding the JSON structure.
You don't need to memorize everything. Learn to find the fields you need.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Better
Engineers
Brighter
Systems

DevOps
Today
A Stronger
Tomorrow

Same
Data
Bigger
Possibilities

JSON with curl

FETCH, SEND AND WORK WITH JSON DATA USING CURL

Simple commands. Real DevOps use cases. Big possibilities.

1 WHAT IS CURL?

- curl is a command-line tool used to transfer data over different network protocols (HTTP, HTTPS, etc.).
- It is widely used in DevOps to interact with APIs.
- You can use curl to fetch data, send data, and work with JSON.

2 FETCHING API DATA

- Use a simple GET request to fetch data from an API.
- Most modern APIs return data in JSON format.

```
curl https://api.github.com/users/veriqta
```

3 USING curl -s

- The **-s** (silent) option hides the progress output and shows only the response data.
- Useful for scripts and clean output.

```
curl -s https://api.github.com/users/veriqta
```

4 HTTP HEADERS

- Use headers to send additional information with your request.
- Common headers:
 - Content-Type
 - Authorization
 - Accept
 - User-Agent

```
curl -H "Accept: application/json" \
https://api.github.com/users/veriqta
```

5 SENDING JSON

- Use the **-d** option to send JSON data in a POST or PUT request.
- Set the **Content-Type** header to **application/json**.

```
curl -X POST https://api.example.com/data \
-H "Content-Type: application/json" \
-d '{"name": "veriqta", "type": "devops"}
```

6 CONTENT-TYPE: application/json

- The **Content-Type** header tells the server that you are sending JSON data.
- Always use:

Content-Type: application/json
- Without this header, the API may reject your request.

7 GET EXAMPLE

Get public information about a user from GitHub API.

```
curl -s https://api.github.com/users/veriqta | jq .
```



JUNIOR ENGINEER TIP

Start with simple GET requests. Read and understand the JSON response. Then try POST requests. Always check the API documentation for required headers and payload format.

8 POST EXAMPLE

Send JSON data to a test API (https://httpbin.org).

```
curl -X POST https://httpbin.org/post \
-H "Content-Type: application/json" \
-d '{"name": "veriqta", "role": "devops"}
```

Expected Response (shortened):

```
{
  "json": {
    "name": "veriqta",
    "role": "devops"
  }
}
```

9 SECURITY REMINDER



- Never expose tokens, passwords, or sensitive keys in screenshots or Git repositories.
- Use environment variables or secret management tools.
- Examples of sensitive data:
 - API tokens
 - Access keys
 - Passwords
 - Private endpoints

KEEP YOUR CREDENTIALS SECURE.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

 Better
Engineers
Brighter
Systems

VERIQTA

LEARN | PRACTICE | BUILD | GROW

 DevOps
Today
A Stronger
Tomorrow

jq THE DEVOPS ENGINEER'S JSON TOOL

POWERFUL. SIMPLE. ESSENTIAL.

Same
Data
Bigger
Possibilities

DevOps
Drives
Progress

1 WHAT jq IS

- jq is a powerful command-line tool for parsing, filtering, and transforming JSON data.
- It takes JSON input and lets you extract the exact information you need.
- It works on Linux, macOS and Windows.
- It is lightweight, fast, and widely used in DevOps, cloud, and automation.



Small Tool
Big Power

2 WHY ENGINEERS USE IT

- API responses are often large and complex.
- jq helps you quickly find and extract specific data.
- It saves time during troubleshooting and automation.
- It can filter, transform, and format JSON.
- It is commonly used with curl, AWS CLI, Kubernetes, and many other tools.
- It makes scripts cleaner and more reliable.

3 PRETTY-PRINTING JSON

- Makes JSON output easier to read.

```
cat data.json | jq .
curl -s https://api.github.com/users/veriqta | jq .
```

4 SELECTING A FIELD

- Extract a specific field from JSON.

```
curl -s https://api.github.com/users/veriqta | jq .login
curl -s https://api.github.com/users/veriqta | jq .id
```

5 NESTED FIELDS

- Access fields inside objects.

```
curl -s https://api.github.com/users/veriqta | \
jq .name
jq .public_repos
jq .owner.login
```

6 WORKING WITH ARRAYS

- Handle arrays and multiple items.

```
curl -s https://api.github.com/users/veriqta/repos | \
jq .[0].name
jq .[1].name
jq '.[ ] | select(.language == "Python") | .name'
```

7 PIPELINES

- Combine commands using pipes for powerful workflows.

```
curl -s https://api.github.com/users/veriqta/repos | \
jq '.[ ] | {name: .name, stars: .stargazers_count}' | \
jq 'sort_by(.stars) | reverse'
```

8 COMPLETE EXAMPLE: curl ... | jq

- Fetch real API data and extract useful information.

```
curl -s https://api.github.com/users/veriqta | \
jq '{
  login: .login,
  name: .name,
  public_repos: .public_repos,
  followers: .followers
}'
```

9 JUNIOR ENGINEER TIP



jq turns large API output into useful information.
Learn to explore and filter JSON with jq.
It will make you faster and more effective in real DevOps work.

"Same Data
Smarter Engineers
Better Results"



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Better
Engineers
Brighter
Systems

VERIQTA

LEARN | PRACTICE | BUILD | GROW

DevOps
Today
A Stronger
Tomorrow

jq FILTERS

YOU WILL ACTUALLY USE

PRACTICAL FILTERS FOR REAL DEVOPS WORK

Same
Data
Bigger
Possibilities

DevOps
Drives
Progress

1 . (PRETTY PRINT)

- The `.` filter outputs the entire JSON in a readable format.
- Useful for quickly viewing API responses.

```
curl -s https://api.github.com/users/veriqta | jq .
```

2 .name (SELECT A FIELD)

- Extract the value of the name field from a JSON object.

```
curl -s https://api.github.com/users/veriqta | jq .name
```

3 .metadata.name (NESTED FIELD)

- Access nested fields inside objects (common in Kubernetes).

```
kubectl get pods -o json | jq .items[0].metadata.name
```

4 .items[] (ARRAY ITEMS)

- Loop through all items in an array.

```
kubectl get pods -o json | jq .items[].metadata.name
```

5 select() (FILTER RESULTS)

- Filter items based on a condition.

```
kubectl get pods -o json | jq '.items[] | select(.status.phase == "Running") | .metadata.name'
```

6 map() (TRANSFORM DATA)

- Apply an operation to each item in an array.

```
kubectl get pods -o json | jq '.items | map(.metadata.name)'
```

7 length (COUNT ITEMS)

- Get the number of items in an array.

```
kubectl get pods -o json | jq '.items | length'
```

8 keys (LIST KEYS)

- Show all keys in a JSON object.

```
curl -s https://api.github.com/users/veriqta | jq 'keys'
```

9 RAW OUTPUT WITH -r

- Use `-r` to output raw strings (without quotes). Useful in scripts.

```
curl -s https://api.github.com/users/veriqta | jq -r .login
```

10 REAL TROUBLESHOOTING EXAMPLES

EXAMPLE 1: FIND A SPECIFIC VALUE

Find the image used by a Kubernetes container.

```
kubectl get pod myapp -o json | jq -r '.spec.containers[].image'
```

EXAMPLE 2: FILTER BY CONDITION

List only failed pods.

```
kubectl get pods -o json | jq -r '.items[] | select(.status.phase == "Failed") | .metadata.name'
```

EXAMPLE 3: EXTRACT AND FORMAT

Get pod names and their status together.

```
kubectl get pods -o json | jq -r '.items[] | "\(.metadata.name)\t\(.status.phase)'
```



JUNIOR ENGINEER TIP

jq turns large API output into useful information. Start with simple filters, practice often, and combine filters to solve real problems.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Better
Engineers
Brighter
Systems

VERIQTA

LEARN | PRACTICE | BUILD | GROW

DevOps
Today
A Stronger
Tomorrow

Same
Data
Bigger
Possibilities

JSON IN AWS CLI

WORK WITH AWS DATA USING JSON

Query • Filter • Automate • Build • Scale



1 AWS CLI RETURNS JSON BY DEFAULT

- AWS CLI returns output in JSON format by default.
- JSON makes it easy to parse, filter, and use in automation.
- You can change the output format, but JSON is recommended for DevOps work.

```
aws sts get-caller-identity
{
  "UserId": "AIDAEXAMPLE123",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/veriqta"
}
```

2 READING EC2 OUTPUT

- Use AWS CLI to get information about EC2 instances.
- The output is in JSON and contains many fields.

```
aws ec2 describe-instances
{
  "Reservations": [
    {
      "Instances": [
        {
          "InstanceId": "i-0abc123def456",
          "InstanceType": "t3.micro",
          "State": {
            "Name": "running"
          }
        }
      ]
    }
  ]
}
```

3 IAM OUTPUT

- Get IAM users, roles, or policies in JSON format.
- Useful for auditing and automation.

```
aws iam list-users
{
  "Users": [
    {
      "UserName": "veriqta",
      "UserId": "AIDAEXAMPLE",
      "Arn": "arn:aws:iam::123456789012:user/veriqta",
      "CreateDate": "2025-01-10T12:00:00Z"
    }
  ]
}
```

4 --output json

- You can explicitly set the output format to JSON using `--output json`.
- This is helpful when you have changed the default output format.

```
aws ec2 describe-instances \
  --output json
```

5 --query

- Use `--query` with JMESPath to extract specific fields.
- This reduces large output to only what you need.

```
aws ec2 describe-instances \
  --query "Reservations[].Instances[0].\
  {ID:InstanceId,Type:InstanceType,\
  State:State.Name}" \
  --output json
```

6 JMESPATH BASICS

- JMESPath is a query language built into AWS CLI.
- Helps you filter and extract data.
- Common examples:

```
Reservations[].Instances[] - List all instances
Instances[0].InstanceId    - First instance ID
Users[].UserName           - All IAM usernames
length(Users)              - Count items
sort_by(Users, &UserName) - Sort results
```

7 COMBINING AWS CLI AND jq

- Use `jq` for more advanced filtering, formatting, or processing.
- Great for scripts and automation.

```
aws ec2 describe-instances --output json | jq \
  '.Reservations[].Instances[] | \
  {id: .InstanceId, type: .InstanceType,\
  state: .State.Name}'
```

8 EXAMPLE: aws ec2 describe-instances

- Full example with filtering and formatting.

```
aws ec2 describe-instances --output json | jq \
  '.Reservations[].Instances[] | \
  { \
    id: .InstanceId, \
    type: .InstanceType, \
    state: .State.Name, \
    az: .Placement.AvailabilityZone, \
    private_ip: .PrivateIpAddress \
  }'
```

9 DEVOPS CONNECTION

- AWS CLI JSON output is used for:
 - Infrastructure automation (Terraform, Ansible)
 - CI/CD pipelines (GitHub Actions, GitLab CI)
 - Monitoring and incident response
 - Auditing and compliance
 - Building custom tools and scripts

JSON makes AWS services programmable. It turns cloud infrastructure into automation.



JUNIOR ENGINEER TIP

Start with simple queries, then combine `--query` and `jq` to get exactly the data you need. Use JSON output in your scripts and pipelines to automate real DevOps tasks.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

 Better
Engineers
Brighter
Systems

VERIQTA

LEARN | PRACTICE | BUILD | GROW

 DevOps
Today
A Stronger
Tomorrow

JSON IN KUBERNETES

KUBERNETES API STORES STRUCTURED OBJECTS



Kubernetes stores everything as structured objects (in JSON format) through its API. When you use `kubectl`, you are interacting with the Kubernetes API, which returns JSON data.

1 KUBERNETES API AND JSON

- Kubernetes stores all resources (Pods, Deployments, Services, etc.) as structured JSON objects.
- `kubectl` communicates with the Kubernetes API and can return the data in JSON format.
- JSON helps tools, scripts, and automation read and process Kubernetes resources.

2 KEY SECTIONS IN A KUBERNETES JSON OBJECT

metadata	Information about the object (name, namespace, labels, annotations, etc.).
spec	Desired state of the object (containers, images, ports, etc.).
status	Current state of the object (running, pending, conditions, etc.).

3 GETTING KUBERNETES RESOURCES AS JSON

```
$ kubectl get pods -o json
```

- This command returns the full JSON representation of all Pods in the current namespace.

4 EXAMPLE: POD JSON (SIMPLIFIED)

```
1 {
2   "apiVersion": "v1",
3   "kind": "Pod",
4   "metadata": {
5     "name": "nginx-pod",
6     "namespace": "default",
7     "labels": {"app": "nginx"}
8   },
9   "spec": {
10    "containers": [{"name": "nginx",
11                     "image": "nginx:1.25"}]
12  },
13   "status": {"phase": "Running"}
14 }
```

5 READING POD JSON

- Look at `metadata.name` to find the Pod name.
- Check `metadata.namespace` for the namespace.
- Inspect `spec.containers` to see the container details.
- Look at `status.phase` to see the current state (Running, Pending, Failed, etc.).
- Use this information to troubleshoot, automate, or extract values with tools like `jq` or `JSONPath`.

6 USING JSONPATH

- JSONPath lets you extract specific fields from JSON output.
- Common examples:

```
$ kubectl get pods -o jsonpath='{.items[*].metadata.name}'
# Get all pod names

$ kubectl get pods -o jsonpath='{.items[*].status.phase}'
# Get pod status

$ kubectl get pod nginx-pod -o jsonpath='{.spec.containers[0].image}'
# Get container image
```

7 TROUBLESHOOTING CONNECTION

- If you get an error, check:
 - Your `kubeconfig` is correct
 - You have access to the cluster
 - The cluster is running
 - You are using the correct context and namespace

COMMON ERRORS

- Unable to connect to server
- Unauthorized
- Forbidden
- The connection was refused
- Context not found



JUNIOR ENGINEER TIP

Don't just look at the output, understand the structure. Learn how metadata, spec, and status work. This will help you read, troubleshoot, and automate Kubernetes resources confidently.



REAL WORLD USE CASES

- Troubleshoot failing Pods
- Extract values for automation scripts
- Monitor cluster state
- Integrate with CI/CD pipelines
- Build custom tools using Kubernetes API data



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Better
Engineers
Brighter
Systems

PRACTICE TODAY. BUILD TOMORROW.

DEVOPS
FOR A
BRIGHTER
TOMORROW

JSON IN TERRAFORM

TERRAFORM STATE, OUTPUT AND AUTOMATION

Terraform uses JSON to represent its state and output in a machine-readable format. This allows automation tools, CI/CD pipelines, and scripts to work with Terraform data.

1 TERRAFORM STATE CONCEPTS

- Terraform state is a record of the real infrastructure that Terraform manages.
- It maps your configuration to actual resources in your cloud environment.
- State helps Terraform know what to create, update, or destroy.
- The state file is stored locally (terraform.tfstate) or in a remote backend (e.g., S3, Terraform Cloud).
- The state file is in JSON format.

Terraform
State File

2 terraform show -json

- Converts the current state or plan into JSON format.
- Provides a detailed, machine-readable view of your infrastructure.
- Useful for automation, analysis, and integrations.

```
$ terraform show -json > state.json
```

This outputs the full state in JSON format which can be parsed by tools like jq.

3 MACHINE-READABLE OUTPUT

- JSON output is easy for machines to parse.
- It contains structured data about resources, attributes, and dependencies.
- Used by automation scripts, CI/CD pipelines, monitoring tools, and custom tooling.

```
1 {
2   "fermat_version": "1.0",
3   "terraform_version": "1.6.0",
4   "values": {
5     "root_module": {
6       "resources": [ ... ]
7     }
8 }
```

4 terraform output -json

- Outputs the defined Terraform outputs in JSON format.
- Useful for scripts and automation to get values such as IP addresses, endpoints, or IDs.

```
$ terraform output -json
1 {
2   "vpc_id": { "value": "vpc-0a1b2c3d" },
3   "instance_ip": { "value": "3.22.11.45" }
4 }
```

5 USING JSON IN AUTOMATION

- Integrate Terraform JSON output with tools like jq, Python, Bash, or CI/CD pipelines.
- Extract specific values for use in deployment scripts.
- Use JSON to validate infrastructure, generate reports, or trigger other actions.

```
$ terraform output -json | jq -r '.instance_ip.value'
3.22.11.45
```



6 WHY AVOID MANUALLY EDITING STATE JSON

- The state file is automatically managed by Terraform.
- Manual changes can corrupt the state file and cause unexpected behaviour.
- It can lead to resources being recreated, modified, or destroyed.
- Always use Terraform commands to make changes, not manual edits.



**DO NOT
EDIT MANUALLY**



JUNIOR ENGINEER TIP

Use Terraform's JSON output to integrate with other tools, but always treat the state file as sensitive. Never edit it manually and always keep it secure.

7 PRODUCTION AWARENESS



- Terraform state can contain sensitive data such as:
 - API keys
 - Passwords
 - Database credentials
 - Private IP addresses
 - Other confidential information.
- Store state securely using remote backends (e.g., AWS S3 with encryption and access controls).
- Restrict access to the state file.
- Treat state files as sensitive production data.



**PROTECT
YOUR STATE
FILE**



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Better
Engineers
Brighter
Systems

VERIQTA

LEARN | PRACTICE | BUILD | GROW

DevOps
Today
A Stronger
Tomorrow

JSON IN CI/CD PIPELINES

PASSING, USING AND AUTOMATING WITH STRUCTURED DATA

JSON is widely used in CI/CD pipelines to pass data, process API responses, store build and deployment metadata, and integrate with tools and services. Understanding how to read and use JSON in pipelines helps you build reliable and automated workflows.

1 PASSING STRUCTURED DATA BETWEEN JOBS

- CI/CD systems pass data between jobs using JSON.
- JSON keeps data structured and easy to read.
- It can include build info, artifact details, deployment results, or custom values.
- Used in environment variables, output files, or pipeline artifacts.

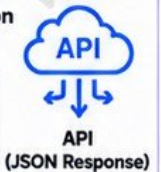


```

{
  "app": "myapp",
  "version": "1.0.0",
  "environment": "staging",
  "status": "success"
}
  
```

2 API RESPONSES

- Many pipelines call external APIs (e.g. GitHub, AWS, Docker, monitoring tools).
- API responses are usually in JSON format.
- You parse the JSON to extract useful information like IDs, URLs, or status.



```

curl -s https://api.github.com/repos/veriqta/demo |
jq '.full_name, .stargazers_count'

"veriqta/demo"
125"
  
```

3 BUILD METADATA

- CI systems generate metadata about the build.
- This can include commit ID, branch name, build number, timestamp, and status.
- Often stored and passed as JSON.



```

{
  "build_number": "102",
  "commit": "a1b2c3d",
  "branch": "main",
  "status": "success",
  "timestamp": "2025-09-04T10:30:00Z"
}
  
```

4 DEPLOYMENT METADATA

- After deployment, pipelines can output metadata about the deployed environment.
- This may include environment name, version, URL, resource IDs, and status.



```

{
  "environment": "production",
  "version": "1.0.0",
  "url": "https://app.veriqta.com",
  "deployment_id": "dep-12345",
  "status": "deployed"
}
  
```

5 ARTIFACT METADATA

- Artifacts (e.g. Docker images, packages, build files) include metadata in JSON format.
- This can include image name, tag, size, checksum, and download URL.



```

{
  "image": "veriqta/myapp",
  "tag": "1.0.0",
  "size": "250MB",
  "digest": "sha256:abc123...",
  "url": "https://registry.veriqta.com/..."
}
  
```

6 READING JSON IN CI/CD TOOLS



```

- name: Get JSON
  id: get_data
  run: |
    echo "DATA=$(curl -s \
    https://api.github.com/...)"
    echo "::set-output \
    name=data::\$DATA"
  
```

```

get_data:
  script:
    - response=
      $(curl -s https://api...)
    - echo "$response" >
      data.json
  artifacts:
    paths:
      - data.json
  
```

```

pipeline {
  stages {
    stage('Get Data') {
      steps {
        sh 'curl -s https://api... \
        -o data.json'
      }
    }
  }
}
  
```

7 USING JSON OUTPUTS IN AUTOMATION

- Parse JSON using tools like jq, Python, or native features in CI/CD tools.
- Use extracted values to make decisions, trigger deployments, create notifications, or generate reports.



```

VERSION=$(cat data.json | jq -r '.version')
echo "Deploying version: $VERSION"
  
```

JSON → Automation

8 COMMON FAILURE: WRONG FIELD PATH

- JSON structures can be nested, and using the wrong field path will return empty or incorrect values.
- Always inspect the JSON output to confirm the correct path.
- Use jq or pretty-printing to understand the structure.



EXAMPLE OF A WRONG FIELD PATH

```
cat data.json | jq -r '.image.tag' # returns nothing
```

Correct path:

```
cat data.json | jq -r '.build.image.tag'
```


JSON IN MONITORING AND OBSERVABILITY

FROM LOGS AND METRICS TO INCIDENT RESPONSE



Monitoring and observability tools use JSON to collect, store, and exchange data such as logs, metrics, alerts, and events. Understanding JSON helps you troubleshoot issues, build integrations, and respond faster during incidents.

1 LOG EVENTS

- Logs are often stored and transferred in JSON format.
- Each log entry contains structured fields.
- Helps with searching, filtering, and analysis.

```
{
  "timestamp": "2025-08-30T12:45:01Z",
  "level": "INFO",
  "service": "payment-service",
  "message": "Payment processed successfully",
  "request_id": "abc123"
}
```



Log
Events

2 ALERT PAYLOADS

- Alerting tools send alerts in JSON format.
- Contains details about the issue, severity, service, etc.
- Used by tools like Prometheus, Grafana, Datadog, PagerDuty, CloudWatch, etc.

```
{
  "alertname": "HighCPUUsage",
  "severity": "critical",
  "service": "api-service",
  "environment": "production",
  "description": "CPU usage is above 90%",
  "startsAt": "2025-08-30T12:50:00Z"
}
```



Alert
Payloads

3 WEBHOOKS

- Many monitoring and incident tools send JSON payloads via webhooks.
- Used to integrate with Slack, Teams, PagerDuty, or custom scripts.
- Webhooks allow real-time notifications and automation.

```
{
  "event": "alert",
  "status": "firing",
  "service": "frontend",
  "url": "https://grafana.com/alert/123",
  "timestamp": "2025-08-30T12:55:00Z"
}
```



Webhooks

4 METRICS APIS

- Metrics systems expose data in JSON through APIs.
- Used by Prometheus, Datadog, CloudWatch, etc.
- You can query metrics, check thresholds, and build dashboards.

```
{
  "status": "success",
  "data": {
    "metric": "http_requests_total",
    "value": 12345,
    "labels": {"service": "api", "env": "prod"}
  }
}
```



Metrics
APIs

5 INCIDENT NOTIFICATIONS

- Incident tools send structured JSON for automated response.
- Contains information like incident ID, service, severity, and status.
- Used for paging, ticket creation, and runbooks.

```
{
  "incident_id": "INC-1001",
  "service": "database",
  "severity": "critical",
  "status": "triggered",
  "timestamp": "2025-08-30T13:00:00Z"
}
```



Incident
Notifications

6 FINDING KEY INFORMATION

- You can extract important fields from JSON using tools like jq or your monitoring platform.
- Common fields to look for:

Service name	service, app, or job
Severity	severity, level
Timestamp	timestamp, startsAt, time
Environment	environment, env, cluster
Message	message, description, summary



Find
Key
Fields

7 DEVOPS CONNECTION: INCIDENT RESPONSE

- JSON provides structured, consistent data for faster troubleshooting.
- Helps automate detection, alerting, and remediation.
- Used in runbooks, dashboards, and incident chatops.
- Understanding JSON improves your ability to respond to real incidents.



Faster
Incident
Response

8 JUNIOR ENGINEER TIP



Learn to read and understand real JSON from your monitoring tools. In incidents, the ability to quickly find the right information saves time and reduces impact.

EDITING AND VALIDATING JSON SAFELY



KEEP YOUR JSON CORRECT, CLEAN AND PRODUCTION READY

JSON is often used in APIs, cloud tools, CI/CD pipelines, configuration files and automation. Editing and validating JSON correctly helps you avoid errors, failed deployments and production issues.

1 JSON VALIDATORS

- Use online or offline JSON validators to check syntax and structure.
- They help you find missing commas, brackets and quotes.
- Popular tools:

- jsonlint.com
- jsonchecker.com
- jq (command line)
- VS Code JSON extension
- JetBrains IDEs



Validate
Before Use

2 PRETTY FORMATTING

- Pretty formatting makes JSON readable for humans.
- Useful for troubleshooting and manual review.
- Use jq to format JSON:

```
$ jq . file.json
```



Readable
and Clean

3 MINIFIED JSON

- Minified JSON removes unnecessary whitespace to make the file smaller.
- Useful for API requests, configuration files and production deployments.
- Use jq to minify JSON:

```
$ jq -c . file.json
```



Small Size
Same Data

4 SYNTAX CHECKING

- Check your JSON syntax before using it in tools, APIs or pipelines.
- Use jq for quick validation:

```
$ jq . file.json
```

```
# If there is no error, the JSON is valid.
# If there is an error, jq will show the exact
# line and issue.
```



Check
Syntax

5 IDE VALIDATION

- Use editors like VS Code, JetBrains or Sublime Text.
- They highlight errors, suggest fixes and format JSON automatically.
- Install JSON extensions for better validation.



VS Code
JSON Tools

6 DIFF BEFORE DEPLOYMENT

- Always compare changes before deploying.
- Helps you see what was added, removed or modified.
- Use standard diff tools:

```
$ diff old.json new.json
$ jq --sort-keys . old.json > sorted_old.json
$ jq --sort-keys . new.json > sorted_new.json
$ diff sorted_old.json sorted_new.json
```



Compare
Changes

7 COMMON PRODUCTION MISTAKE

- Manually editing large JSON files can easily introduce syntax errors.
- It can also lead to accidental deletion or modification of important data.
- Always use proper tools and validation.



Avoid Manual
Editing

8 VERIFY BEFORE USING IN PIPELINES

- Validate JSON before using it in CI/CD pipelines, APIs or automation scripts.
- Ensure the structure and values are correct.
- This prevents pipeline failures and production issues.



Verify
and Deploy



JUNIOR ENGINEER TIP

Always validate, format and review your JSON. Small mistakes can cause big problems in automation, deployments and production environments.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Practical
Skills
Real
Results

VERIQTA

LEARN | PRACTICE | BUILD | GROW

Same
Data
Bigger
Possibilities

TROUBLESHOOTING BROKEN JSON IN DEVOPS

Common JSON errors, how to find them, fix them, and get your pipelines, APIs, and configurations working again.

1 PARSE ERRORS

- JSON cannot be parsed because the format is invalid.

```
1 {
2   "name": "veriqta"
3   "type": "devops"
4 }
```

 Error: Unexpected end of JSON input (missing closing brace)

2 UNEXPECTED TOKEN

- An invalid character was found in the JSON.

```
1 {
2   "name": "veriqta",
3   "role": devops,
4 }
```

 Error: Unexpected token d in JSON at position 32

3 MISSING COMMA

- Each key-value pair must be separated by a comma.

```
1 {
2   "name": "veriqta"
3   "type": "devops"
4 }
```

 Error: Expected ',', '}' or '}' after property value

4 MISSING QUOTE

- Keys and string values must be in double quotes.

```
1 {
2   name: "veriqta",
3   "type": "devops"
4 }
```

 Error: Unexpected token n in JSON at position 4

5 WRONG DATA TYPE

- The value type does not match what is expected.

```
1 {
2   "port": "8080",
3   "enabled": "true"
4 }
```

 Issue: port should be a number and enabled should be a boolean (true, not "true").

6 WRONG NESTING LEVEL

- Braces or brackets are in the wrong place.

```
1 {
2   "spec": {
3     "containers": [
4       { "name": "nginx" }
5     ]
6   }
```

 Error: Unexpected token '}' in JSON at position 120

7 MISSING FIELD

- A required field is not present in the JSON.

```
{
  "name": "veriqta",
  "type": "devops"
}
```

 Error: Missing required field (e.g. "image" in Kubernetes spec)

8 NULL VALUES

- A field has a null value when it should have a real value.

```
{
  "name": "veriqta",
  "image": null,
  "port": null
}
```

 Issue: image and port cannot be null in this context.

COMMON PLACES THIS HAPPENS

- API requests and responses
- Kubernetes manifests
- Terraform state and outputs
- CI/CD pipeline configurations
- Cloud CLI outputs (AWS, Azure, GCP)
- Monitoring and alert payloads
- Automation scripts
- Configuration files



9 DIAGNOSTIC WORKFLOW

Follow these steps to troubleshoot any JSON issue.



1. ERROR

Read the error message carefully.



2. VALIDATE

Use a JSON validator or jq.



3. LOCATE

Find the exact line or position of the issue.



4. FIX

Correct the syntax or value.



5. RE-RUN

Test again in your tool, API, or pipeline.



6. VERIFY

Confirm the JSON is valid and the process works.



JUNIOR ENGINEER TIP

Read the structure. Most JSON issues are simple: a missing comma, quote, or bracket. Validate early to save time and avoid pipeline failures.



REMEMBER

A small JSON syntax error can break a deployment, fail a pipeline, or stop an integration. Always validate your JSON before using it in production.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

VERIQTA

LEARN | PRACTICE | BUILD | GROW

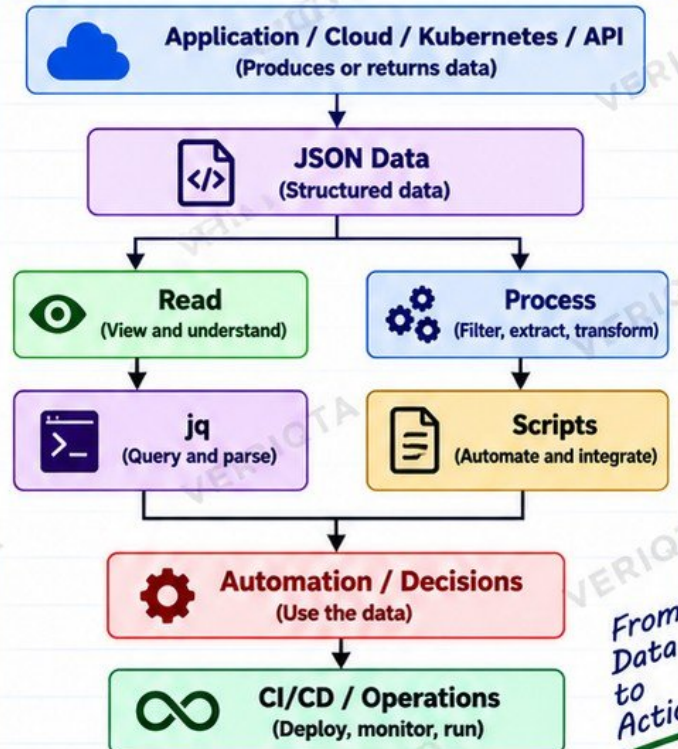
FINAL DEVOPS JSON LAB

From JSON Fundamentals to Real DevOps Automation and Configuration

1 LAB OBJECTIVES

- 1 Fetch JSON from an API
- 2 Pretty-print it
- 3 Find nested fields
- 4 Filter an array
- 5 Extract one value
- 6 Save JSON to a file
- 7 Validate the file
- 8 Use curl to send JSON
- 9 Read AWS CLI JSON
- 10 Read Kubernetes JSON
- 11 Parse output with jq
- 12 Troubleshoot one deliberately broken JSON document
- 13 Verify corrected output

2 FINAL WORKFLOW



3 USEFUL COMMAND EXAMPLES

```

1 # Fetch JSON from an API
2 curl -s https://api.github.com/users/veriqta | jq .
3 # Pretty-print JSON
4 cat data.json | jq .
5 # Find a nested field
6 kubectl get pod nginx -o json | jq '.metadata.name'
7 # Filter an array
8 aws ec2 describe-instances --output json | jq
  '.Reservations[] | .Instances[] | {id: .InstanceId,
  state: .State.Name}'
  
```

4 SUCCESS CHECKLIST

- ☒ I can fetch JSON from an API
- ☒ I can pretty-print and read it
- ☒ I can find nested fields
- ☒ I can filter arrays and extract values
- ☒ I can validate JSON files
- ☒ I can use curl to send JSON
- ☒ I can read AWS CLI JSON
- ☒ I can read Kubernetes JSON
- ☒ I can parse output with jq
- ☒ I can troubleshoot and fix broken JSON
- ☒ I can verify the corrected output



Small
Steps
Real
Progress

5 JUNIOR ENGINEER TIP



Practice with real data from APIs, cloud tools, and Kubernetes. The more JSON you read and work with, the faster you will become confident.

6 REMEMBER



JSON is everywhere in DevOps. Learn to read it, validate it, and use it. This skill will save you time and help you solve real production problems.